

Data Structures in R

Richard Röttger

BMB547

2024

The Red Thread of the Course

BMB547 · Mathematics for the Life Sciences (SDU)

The red thread: learn practical math & data tools for life science

Teacher: JB

VS

RR

VS

KD

RR

KD

JB/KD



Math functions &
calculus

Intro to R + data
viz

Data structures (R)

Descriptive
statistics

Linear regression

Multiple linear
regression

Linear algebra
(matrices)

Bio applications of
derivative

Today: Lecture 3 of 8

Table of Contents

- What are Data Structures?
- Data Structures in R
 - Atomic Data Types
 - Working with Logic
 - Vectors
 - Lists
 - Factors
 - Matrices
 - Dataframes
- Loading Your Data

➤ The slides are based on https://evol.bio.lmu.de/_statgen/Rcourse/ws1617/

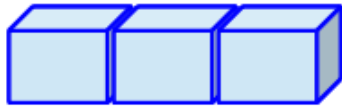
What is a data structure?

- In order to perform computations, we need to access and process data
- The data in the computer needs to be hold in memory
- This is achieved by **data structures**:
 - They should be efficient
 - Allow modification too the data
 - Organize access
- **A data structure is used to store and organize data. It is a way of arranging data on a computer so that it can be accessed and updated efficiently.**

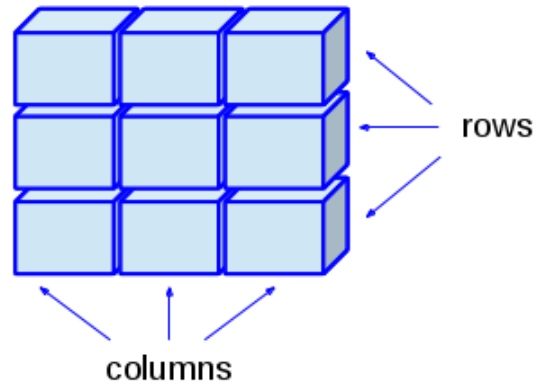
➤ Image: Schork, N. J. Personalized medicine: Time for one-person trials. Nature News 520, 609–611 (2015).

Data Structures in R

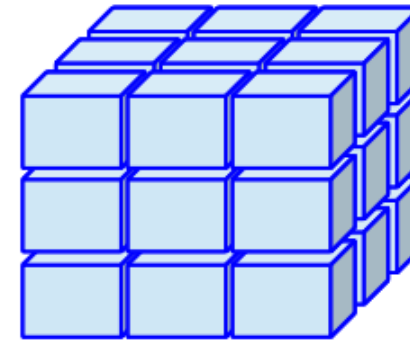
Vector



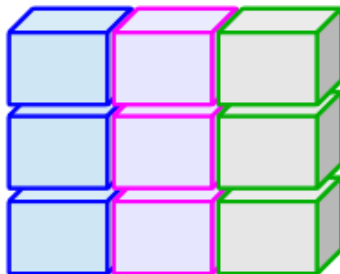
Matrix



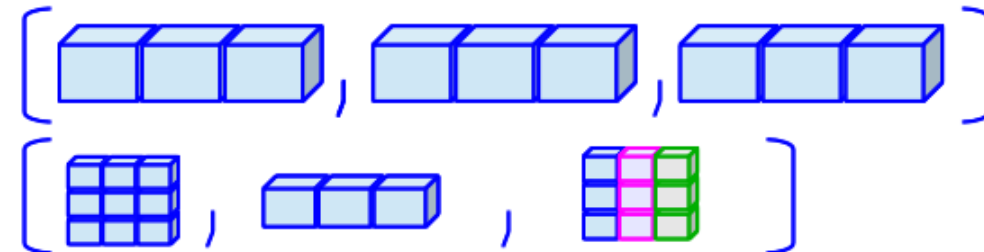
Array



Data Frame
(Table)



Lists



Atomic Datatypes

- R has built in six different atomic data types:
 - Logical
 - Integer
 - Real
 - String (or Character)
 - Complex (not of relevance for us)
 - Raw (not of relevance for us)

Datatype	Description	Example
logical	True or False	TRUE, FALSE
numeric / integer	Whole numbers	1L, 7L, -5L
numeric / double	real numbers	2.3, 3.14, 5
complex	Complex numbers	2.1+3i, 5 + 0i
character	Character or string	"Hi All", "Awesome"

Atomic Datatypes

➤ R has built in six different atomic data types:

➤ **Logical**

- Represents the logical values True and False
- Conditions are evaluated to logical data types

➤ Integer

➤ Real

➤ String (or Character)

➤ Complex (not of relevance for us)

➤ Raw (not of relevance for us)

```
> a <- TRUE
> a
[1] TRUE
> 4 < 5
[1] TRUE
> b <- 4 < 5
> class(b)
[1] "logical"
```

Atomic Datatypes

- R has built in six different atomic data types:
 - Logical
 - **Integer**
 - **Real (Numeric)**
 - Holding Numbers
 - If not explicitly specified, numeric is used
 - Integer is automatically converted to Real, if necessary
 - Forcing to integer will cut-off the non-integer part
 - String (or Character)
 - Complex (not of relevance for us)
 - Raw (not of relevance for us)

```
> a <- 5
> class(a)
[1] "numeric"

> a <- as.integer(5)
> class(a)
[1] "integer"

> b <- 10.7
> c = a+b
> class(b)
[1] "numeric"

> c
[1] 15.7

> as.integer(c)
[1] 15
```

Atomic Datatypes

- R has built in six different atomic data types:
 - Logical
 - Integer
 - Real (Numeric)
 - **String (or Character)**
 - Complex (not of relevance for us)
 - Raw (not of relevance for us)

```
> text = "Hallo Students"
> class(text)
[1] "character"
> number = 8
> class(number)
[1] "numeric"

> as.character(number)
[1] "8"
> as.integer("5")
[1] 5
> as.integer("that won't work")
[1] NA
Warning message:
NAs introduced by coercion
```

Examining Data Types

- Everything is an Object in R
- R provides many functions to examine objects:
 - `class()` - what kind of object is it (high-level)?
 - `typeof()` - what is the object's data type (low-level)?
 - `length()` - how long is it?
 - `attributes()` - does it have any metadata?

Basic Logic Operations

Operator	Executions
>	Greater than
>=	Greater than or equal
<	Less than
<=	Less than or equal
==	Exactly equal to
!=	Not equal to
!	Negation (not)
&, &&	and
,	or

Simple Examples

```
> x <- 7  
> y <- 3  
> x < y  
[1] FALSE
```

```
> x <- 6  
> (x < 9) && (x > 3)  
[1] TRUE
```

```
> x = 20  
> y = 30  
> (x == 20) & (y == 30)  
[1] TRUE
```

```
> x = 20  
> x != 0  
> [1] TRUE
```

- Those operations will become very handy later, when we want to select, e.g., all elements which are larger than x in our dataset

Constructing atomic vectors

- Atomic vectors are constructed in various ways
 - `c()` - function to combine individual values
 - `seq()` - to create more complex sequences
 - `rep()` - to create replicates of values
 - As result from other functions, e.g.
`runif(3, min=0, max=100)`
- They are always flat:
 - You can use other vectors in the constructor
 - Result will be a normal vector again

```
x <- c(1, 2, 3)
> x
[1] 1 2 3

> is.atomic(x)
[1] TRUE
> is.list(x)
[1] FALSE

> y <- c(x, x, 4, 5, 6)
> y
[1] 1 2 3 1 2 3 4 5 6
> c(x,y, c(7, 8, y, 9, c(10, 11)))
[1] 1 2 3 1 2 3 1 2 3 4 5 6 7
8 1 2 3 1 2 3 4 5 6 9 10 11
```

Using seq() and rep()

```
> seq(from=1, to=8)
[1] 1 2 3 4 5 6 7 8

> seq(from=4, to=10, by=2)
[1] 4 6 8 10

> seq(from=1, to=10, length=4)
[1] 1 4 7 10

> 1:8
[1] 1 2 3 4 5 6 7 8
```

```
> rep(1, 4)
[1] 1 1 1 1

> rep(4:5, 3)
[1] 4 5 4 5 4 5

> rep(1:4, each = 2)
[1] 1 1 2 2 3 3 4 4

> rep(1:4, mes=2, each=2)
[1] 1 1 2 2 3 3 4 4
```

Vectors of Different Types

```
> x <- c(1, 2, 3)
> length(x)
[1] 3
> typeof(x)
[1] "double"
> is.atomic(x)
[1] TRUE
> is.numeric(x)
[1] TRUE
> is.double(x)
[1] TRUE
> as.integer(x)
[1] 1 2 3
> x = as.integer(x)
> typeof(x)
[1] "integer"
```

```
> x <- c(T, F, TRUE, T, FALSE, T)
> x
[1] TRUE FALSE TRUE TRUE FALSE TRUE
> length(x)
[1] 6
> typeof(x)
[1] "logical"

> is.atomic(x)
[1] TRUE
> is.numeric(x)
[1] FALSE
> is.logical(x)
[1] TRUE
```

Type Contagion

- Be careful when mixing datatypes.
- From least to most flexible vector types are:
 - Logical
 - Integer
 - Double
 - Complex
 - Character

```
> typeof(c(FALSE, T))
[1] "logical"

> typeof(c(FALSE, T))
[1] "logical"

> typeof(c(FALSE, T, 2L))
[1] "integer"

> typeof(c(FALSE, T, 2L, 3))
[1] "double"

> typeof(c(FALSE, T, 2L, 3, "four"))
[1] "character"

> c(FALSE, T, 2L, 3, "four")
[1] "FALSE" "TRUE"  "2"     "3"     "four"
```



Coercing Vectors

- Like atomic structures, you can coerce vectors
- “as.class_name”
 - as.numeric(x)
 - as.logical(x)
 - as.integer(x)
 - as.character(x)
 - as.complex(x)
 - as.factor(x)
- Be careful: Coercing vectors can introduce NAs.
- For logical conversion, no warning!

```
> x = c(FALSE, T, 2L, 3, "four")

> as.logical(x)
[1] FALSE TRUE  NA  NA  NA

> as.integer(x)
[1] NA NA  2  3 NA
Warning message:
NAs introduced by coercion

> as.numeric(x)
[1] NA NA  2  3 NA
Warning message:
NAs introduced by coercion

> as.complex(x)
[1] NA NA 2+0i 3+0i NA
Warning message:
NAs introduced by coercion
```

Accessing Elements

- Elements of an atomic vector are accessed using the `[]` operator
- Important: R starts numbering at 1!
- You can give a vector with indices to access multiple elements at the same time

```
> x <- c("a", "b", "c", "d", "e", "f")

> x[3]
[1] "c"

> x[c(1,3,5)]
[1] "a" "c" "e"

> x[1] <- "EH"
> x
[1] "EH" "b"  "c"  "d"  "e"  "f"

> x[c(3,5)] <- c("third", "fifth")
> x
[1] "EH"      "b"      "third" "d"      "fifth"
" f"
```

Selecting Elements based on criteria

- You can use logic operations to select only certain elements of a vector

```
> v = c(1,9,5,3,6,2,8,6,9,4,7,0,4,1,7)
> v > 6
[1] FALSE TRUE FALSE FALSE FALSE FALSE TRUE FALSE TRUE FALSE TRUE FALSE FALSE FALSE TRUE
> v > 3 & v < 7
[1] FALSE FALSE TRUE FALSE TRUE FALSE FALSE TRUE FALSE TRUE FALSE FALSE TRUE FALSE FALSE
```

- You can also directly select the matching elements

```
> v[v > 3 & v < 7]
[1] 5 6 6 4 4
```

- If you know at what position the matching elements are, you can use the **which** instruction

```
> which(v > 3 & v < 7)
[1] 3 5 8 10 13
```

Vector Arithmetic

- Vector arithmetic allows the same operation to be performed on all the elements of the vector with a single line, not requiring loops
- Operations are performed element-by-element

```
> a = c(1, 2, 3, 4)
> b = c(5, 6, 7, 8)

> 5 * a
[1] 5 10 15 20

> 5 * b
[1] 25 30 35 40

> a + b
[1] 6 8 10 12

> a * b
[1] 5 12 21 32

a / b
[1] 0.2000000 0.3333333 0.4285714
0.5000000
```

Naming Vectors

```
> x = c(1,2,3,4,5)
> attributes(x)
NULL

> names(x) = letters[1:5]
> attributes(x)
$names
[1] "a" "b" "c" "d" "e"

> x
a b c d e
1 2 3 4 5
```

Factors

- A factor is a special kind of atomic vector having class factor and an attribute called levels
- A factor is typically used to represent a categorical variate having a fixed, finite, and known set of values.
- The possible values are assigned to the factor levels.
- The levels are “fixed”

```
> treatment <- factor(c("a", "b", "b", "b", "a", "c", "c", "a",  
"b"))  
> levels(treatment)  
[1] "a" "b" "c"  
  
> table(treatment)  
treatment  
a b c  
3 4 2  
  
> treatment[3] <- "d"  
Warning message:  
In `[<-.factor`(`*tmp*`, 3, value = "d") :  
  invalid factor level, NA generated  
  
> treatment  
[1] a      b      <NA> b      a      c      c      a      b  
Levels: a b c
```

Working with Factors

- A factor could be coerced to a vector
 - `as.vector()`
 - Using the `c()` function
- But loses the level information

```
> answer <- factor(c(1, 2, 1, 1, 2, 2, 1, 1, 2), levels = c(1, 2, 3))
> table(answer)
answer
1 2 3
5 4 0

> as.vector(answer)
[1] "1" "2" "1" "1" "2" "2" "1" "1" "2"
> typeof(answer)
[1] "integer"
```

Lists

- List are very flexible data structures
- Can mix any type of data together
- It is basically a collection of data structures
- The operation `str` show a description of the list
- Almost all functions (e.g., t-test, linear regression, etc.) in R produce output that is stored in a list

```
> x <- list("a", c(2, 3, 4), c(T,F), c("b", "c", "d", 56))
> x
[[1]]
[1] "a"

[[2]]
[1] 2 3 4

[[3]]
[1] TRUE FALSE

[[4]]
[1] "b" "c" "d" "56"

> str(x)
List of 4
 $ : chr "a"
 $ : num [1:3] 2 3 4
 $ : logi [1:2] TRUE FALSE
 $ : chr [1:4] "b" "c" "d" "56"
```

Accessing Elements

- Lists are recursive
 - The element of a list is a list for itself
- Use the `[[ ]]` operator to directly access the data

```
#> x <- list("a", c(2, 3, 4), c(T,F), c("b", "c", "d", 56))

> x[2]
[[1]]
[1] 2 3 4

> typeof(x[2])
[1] "list"
> length(x[2])
[1] 1

> x[[2]]
[1] 2 3 4
> typeof(x[[2]])
[1] "double"
```

Recursive Lists

- Lists are recursive
 - The element of a list is a list for itself
- Use the `[[ ]]` operator to directly access the data

```
> x <- list("one", 2)
> is.recursive(x)
[1] TRUE

> y <- list(x, list(3, x))
> is.recursive(y)
[1] TRUE

> str(y)
List of 2
 $ :List of 2
  ..$ : chr "one"
  ..$ : num 2
 $ :List of 2
  ..$ : num 3
  ..$ :List of 2
  .. ..$ : chr "one"
  .. ..$ : num 2
```

Names in Lists

- The same with vectors, names in lists can be super useful
- You can use the `names()` function, or use the names in the constructor

```
> x <- list(a=c(1,2,3), b=4, 5)
> str(x)
List of 3
 $ a: num [1:3] 1 2 3
 $ b: num 4
 $   : num 5

> x["b"]
$b
[1] 4

> x$b
[1] 4

> names(x)
[1] "a" "b" ""
```

Matrix

- Data structures holding the same type
- Two dimensional arranged in rows and columns
- Can be created:
 - `matrix()` - creates a matrix by specifying rows and columns
 - `dim()` - sets dimensions to a vector
 - `cbind()` - combines columns or rows
 - `rbind()`

```
> matrix(data = 1:6, nrow = 2, ncol = 3)
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

> x <- 1:6
> dim(x) <- c(2, 3)
> x
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

> x <- c(1, 2, 3)
> y <- 10:12
> cbind(x, y)
      x  y
[1,] 1 10
[2,] 2 11
[3,] 3 12
```

Arrays

- Direct extensions to high-dimensional cases
- Can be constructed like matrices
 - `dim()`
 - `array()`

```
> array(1:18, dim=c(3,3,2))
, , 1
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9

, , 2
     [,1] [,2] [,3]
[1,]   10   13   16
[2,]   11   14   17
[3,]   12   15   18
```

Dataframe

- A data frame is a list of equal-length vectors:
 - Results in a table or a two-dimensional array-like structure in which each column contains values of one variable and each row contains one set of values from each column
- Characteristics of a data frame:
 - The column names should not be empty
 - The row names should be unique
 - The data stored in a data frame may be numeric, factor or character
 - Each column contains same number of data items.

Creating Dataframes

- Constructor: `data.frame()`
 - Takes vectors of the same length
 - Strings are automatically converted to factors

```
> df = data.frame(log=c(T,F,T), num=1:3,  
str=c("a", "b", "c"))  
> df  
      log num str  
1  TRUE   1  a  
2 FALSE   2  b  
3  TRUE   3  c  
  
> str(df)  
'data.frame':   3 obs. of  3 variables:  
 $ log: logi  TRUE FALSE TRUE  
 $ num: int   1 2 3  
 $ str: chr  "a" "b" "c"
```

Working with Dataframes



```
> data(iris)
> summary(iris)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
Min. :4.300	Min. :2.000	Min. :1.000	Min. :0.100	setosa :50
1st Qu.:5.100	1st Qu.:2.800	1st Qu.:1.600	1st Qu.:0.300	versicolor:50
Median :5.800	Median :3.000	Median :4.350	Median :1.300	virginica :50
Mean :5.843	Mean :3.057	Mean :3.758	Mean :1.199	
3rd Qu.:6.400	3rd Qu.:3.300	3rd Qu.:5.100	3rd Qu.:1.800	
Max. :7.900	Max. :4.400	Max. :6.900	Max. :2.500	



Working with Dataframes

```
> head(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4          0.2  setosa
2          4.9         3.0          1.4          0.2  setosa
3          4.7         3.2          1.3          0.2  setosa
4          4.6         3.1          1.5          0.2  setosa
5          5.0         3.6          1.4          0.2  setosa
6          5.4         3.9          1.7          0.4  setosa

> tail(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
145          6.7         3.3          5.7          2.5 virginica
146          6.7         3.0          5.2          2.3 virginica
147          6.3         2.5          5.0          1.9 virginica
148          6.5         3.0          5.2          2.0 virginica
149          6.2         3.4          5.4          2.3 virginica
150          5.9         3.0          5.1          1.8 virginica
```

Working with Dataframes

```
> dim(iris)
[1] 150  5
> dimnames(iris)
[[1]]
 [1] "1"  "2"  "3"  "4"  "5"  "6"  "7"  "8"  "9"  "10" "11" "12" "13" "14"
"15" "16" "17" "18" "19" "20" "21" "22" "23" "24" "25" "26"
[27] "27" "28" "29" "30" "31" "32" "33" "34" "35" "36" "37" "38" "39" "40"
"41" "42" "43" "44" "45" "46" "47" "48" "49" "50" "51" "52"
[53] "53" "54" "55" "56" "57" "58" "59" "60" "61" "62" "63" "64" "65" "66"
"67" "68" "69" "70" "71" "72" "73" "74" "75" "76" "77" "78"
[79] "79" "80" "81" "82" "83" "84" "85" "86" "87" "88" "89" "90" "91" "92"
"93" "94" "95" "96" "97" "98" "99" "100" "101" "102" "103" "104"
[105] "105" "106" "107" "108" "109" "110" "111" "112" "113" "114" "115" "116" "117" "118"
"119" "120" "121" "122" "123" "124" "125" "126" "127" "128" "129" "130"
[131] "131" "132" "133" "134" "135" "136" "137" "138" "139" "140" "141" "142" "143" "144"
"145" "146" "147" "148" "149" "150"

[[2]]
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

Accessing Data

- General Access:
 - Use the [] operator
 - You need to specify row and column
 - If you leave one out, it means you take all of them
- Indexing:
 - Indexing by integer vector df[c(1,7,8)]
 - Indexing by name: You can index an element by name using the \$ notation
 - Indexing by logical vector: df[df\$num>1,]

```
> iris$Sepal.Length
[1] 5.1 4.9 4.7 4.6 5.0 5.4 4.6 5.0 4.4 4.9 ...
[40] 5.1 5.0 4.5 4.4 5.0 5.1 4.8 5.1 4.6 5.3 ...
[79] 6.0 5.7 5.5 5.5 5.8 6.0 5.4 6.0 6.7 6.3 ...
[118] 7.7 7.7 6.0 6.9 5.6 7.7 6.3 6.7 7.2 6.2 ...
> iris[3,1]
[1] 4.7
> iris[3,"Sepal.Length"]
[1] 4.7
> iris[3,]
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
3           4.7          3.2          1.3          0.2   setosa
> iris[1,]
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           5.1          3.5          1.4          0.2   setosa
> iris[,1]
[1] 5.1 4.9 4.7 4.6 5.0 5.4 4.6 5.0 4.4 4.9 ...
[40] 5.1 5.0 4.5 4.4 5.0 5.1 4.8 5.1 4.6 5.3 ...
[79] 6.0 5.7 5.5 5.5 5.8 6.0 5.4 6.0 6.7 6.3 ...
[118] 7.7 7.7 6.0 6.9 5.6 7.7 6.3 6.7 7.2 6.2 ...
```

Manipulating Data

- Values can be written in dataframes as in other data structures
- Recycling rule applies if less values are supplied than selected (meaning, they get repeated)
- Be careful, you easily override an entire row or column

```
> iris$Sepal.Length = 1
> head(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           1         3.5         1.4         0.2   setosa
2           1         3.0         1.4         0.2   setosa
3           1         3.2         1.3         0.2   setosa
4           1         3.1         1.5         0.2   setosa
5           1         3.6         1.4         0.2   setosa
6           1         3.9         1.7         0.4   setosa
```

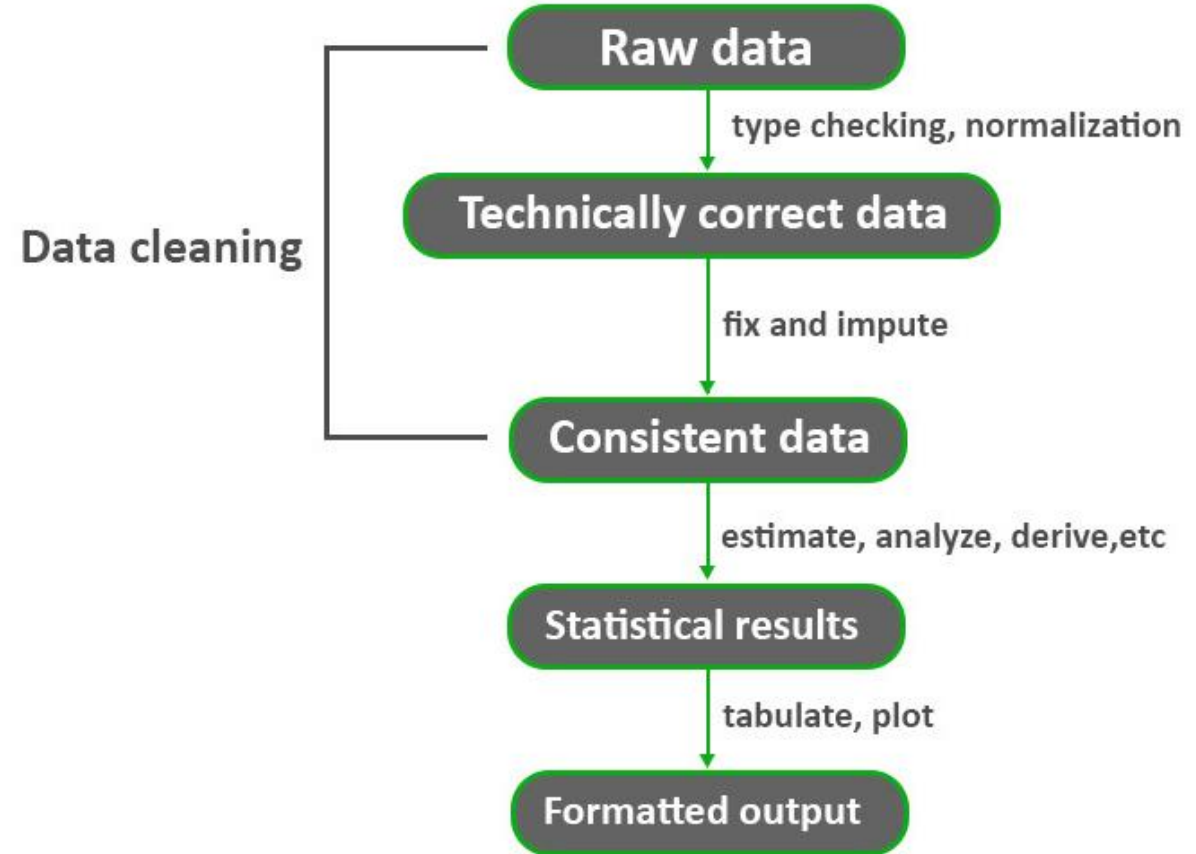
Loading your Data

- Creating your data frames by hand is tiresome and error-prone
- Normally you load your data from a file
- R also ships with plenty of standard datasets
 - You can load those with the `data()` command
 - For instance `data(iris)` creates a data frame with the iris dataset
- Procedure when loading your data:
 - Import your data
 - **Check, clean and prepare your data (might be your main work)**
 - Conduct your analyses
 - Export your results

Data Cleaning Process

➤ Purpose of Data Cleaning

- Eliminate Errors
- Eliminate Redundancy
- Increase Data Reliability
- Delivery Accuracy
- Ensure Consistency
- Assure Completeness
- Standardize your approach



➤ <https://www.geeksforgeeks.org/data-cleaning-in-r/>

How should data look like?

- Columns should contain variables
- Rows should contain observations, measurements, cases, etc.
- Use first row for the names of the variables
- Enter NA (in capitals) into cells representing missing values
- You should avoid names (or fields or values) that contain spaces
- Store data as .csv or .txt files as those can be easily read into R

Read Data

```
read.csv(file, header=FALSE, sep=" ", quote="\'",  
         dec=".", numerals=c("allow.loss", "warn.loss", "no.loss"),  
         row.names, col.names, as.is=!stringsAsFactors, na.strings="NA",  
         colClasses=NA, nrow=-1, skip=0, check.names=TRUE, fill=!blank.lines.skip,  
         strip.white=FALSE, blank.lines.skip=TRUE, comment.char="#",  
         allowEscapes=FALSE, flush=FALSE, stringsAsFactors=default.stringsAsFactors(),  
         fileEncoding="", encoding="unknown", text, skipNul=FALSE)
```

- The `header= TRUE` argument tells R that the first row of your file contains the variable names
- The `sep= ", "` argument tells R that fields are separated by comma
- The `strip.white = TRUE` argument removes white space before or after factors that has been mistakenly insert during data entry (e.g. "small" vs. "small_" become both "small")
- The `na.strings = " "` argument replaces empty cells by NA (missing data in R)

Typical Read Command

```
myData <- read.csv(file = "datafile.csv",  
  header = TRUE,  
  sep = ",",  
  strip.white = TRUE,  
  na.strings = " ")
```

- If you get an error like:

Error in file(file, "rt") : cannot open the connection

In addition: Warning message:

In file(file, "rt"):

cannot open file 'datafile.csv': No such file or directory

- Set your working directory (setwd()) first, so that R uses the right folder to look for your data file!

Lets walk through a typical example

```
myData <- myData<- read.csv(file = "https://environmentalcomputing.net/datasets/Snail_feeding.csv",  
  header = TRUE,  
  sep = ",",  
  strip.white = TRUE,  
  na.strings = "")
```

#sdu.dk

- This loads a small dataset. If you want to find nice tips and tricks and learn more about this dataset, visit <https://environmentalcomputing.net>
- We use a header, set the separator, we ensure that white-spaces are trimmed and indicate that missing values are indicated by a space

First Inspection

```
> str(myData)
'data.frame':   768 obs. of  12 variables:
 $ Snail.ID: int   1 1 1 1 1 1 1 1 1 1 ...
 $ Sex      : chr   "male" "male" "males" "male" ...
 $ Size     : chr   "small" "small" "small" "small" ...
 $ Feeding  : logi   FALSE FALSE FALSE FALSE FALSE TRUE ...
 $ Distance: chr   "0.17" "0.87" "0.22" "0.13" ...
 $ Depth    : num   1.66 1.26 1.43 1.46 1.21 1.56 1.62 162 1.96 1.93 ...
 $ Temp     : int   21 21 18 19 21 21 20 20 19 19 ...
 $ X        : logi   NA NA NA NA NA NA ...
 $ X.1      : logi   NA NA NA NA NA NA ...
 $ X.2      : logi   NA NA NA NA NA NA ...
 $ X.3      : logi   NA NA NA NA NA NA ...
 $ X.4      : logi   NA NA NA NA NA NA ...
```

Remove columns and inspect again

```
> myData = myData[,1:7]
> str(myData)
'data.frame': 768 obs. of 7 variables:
 $ Snail.ID: int 1 1 1 1 1 1 1 1 1 1 ...
 $ Sex      : chr "male" "male" "males" "male" ...
 $ Size     : chr "small" "small" "small" "small" ...
 $ Feeding  : logi FALSE FALSE FALSE FALSE TRUE ...
 $ Distance: chr "0.17" "0.87" "0.22" "0.13" ...
 $ Depth    : num 1.66 1.26 1.43 1.46 1.21 1.56 1.62 162 1.96 1.93 ...
 $ Temp     : int 21 21 18 19 21 21 20 20 19 19 ...
```

- We immediately see a few things:
 - There are two columns which should be a factor
 - We see one column which should not be “chr” but rather “num”

Convert to the correct values

```
> as.numeric(myData$Distance)
[1] 0.17000 0.87000 0.22000 0.13000 0.36000 0.84000 0.69000 0.60000 0.85000 0.59000
0.05000
...
Warning message:
NAs introduced by coercion
```

- Okay, let's find out what's going on here, we are introducing a couple of NAs, probably the reason why we couldn't convert them to numerical values in the first place.
- For that, we can use the function `which()` together with `is.na()` to find the perpetrators!

Correct the values

```
> myData[which(is.na(as.numeric(myData$Distance))),]  
  Snail.ID    Sex  Size Feeding Distance Depth Temp  
682      15 female large   TRUE   0.58'  1.21   NA  
755      16 female large  FALSE 0,356452  1.45   20  
Warning message:  
NAs introduced by coercion
```

- Aha! That is clearly a typo, one should be just 0.58, the other should be 0.356452. Now we can complete the conversion!

```
> myData$Distance[682] = 0.58  
> myData$Distance[755] = 0.356452  
> myData$Distance = as.numeric(myData$Distance)
```

Onwards with the next Entries

```
> myData$Size = as.factor(myData$Size)
> myData$Sex = as.factor(myData$Sex)
```

- That was easy. Check the levels, if everything looks like it is supposed to look

```
> levels(myData$Size)
[1] "large" "small"
> levels(myData$Sex)
[1] "female" "female s" "male" "Male" "males"
```

- There is always something, isn't there. Let's use the which-keyword again and correct the factor!

Onwards with the next Entries

```
> myData$Sex[which(myData$Sex == "female s")] = "female"  
> myData$Sex[which(myData$Sex == "Male")] = "male"  
> myData$Sex[which(myData$Sex == "males")] = "male"  
> levels(myData$Sex)  
[1] "female" "female s" "male" "Male" "males"
```

- Argh, right, the levels are still here. We can remove them, by factoring again!

```
> myData$Sex = factor(myData$Sex)  
> levels(myData$Sex)  
[1] "female" "male"
```

- Much better!

Now, let's look at the data again!

```
> summary(myData)
```

Snail.ID	
Min.	: 1.00
1st Qu.:	4.75
Median :	8.50
Mean :	8.50
3rd Qu.:	12.25
Max.	:16.00

Sex	
female:	384
male :	384

Size	
large:	383
small:	384
NA's :	1

Feeding	
Mode :	logical
FALSE:	502
TRUE :	266

Distance	
Min.	:0.0000
1st Qu.:	0.2775
Median :	0.5100
Mean :	0.5122
3rd Qu.:	0.7500
Max.	:1.0000

Depth	
Min.	: 1.000
1st Qu.:	1.260
Median :	1.510
Mean :	1.716
3rd Qu.:	1.760
Max.	:162.000

Temp	
Min.	:18.00
1st Qu.:	19.00
Median :	19.00
Mean :	19.49
3rd Qu.:	20.00
Max.	:21.00
NA's :	6

➤ We see a few more things:

- The IDs should be rather a factor (`myData$Snail.ID = as.factor(myData$Snail.ID)`)
- We have a few NA left
- We have a blatant outlier!

Further cleaning

```
> myData[which(is.na(myData$Size)),]  
  Snail.ID    Sex Size Feeding Distance Depth Temp  
741      16 female <NA>   FALSE    0.55  1.37   18  
> myData[myData$Snail.ID == 16,]  
  Snail.ID    Sex Size Feeding Distance Depth Temp  
721      16 female large   FALSE 0.780000  1.04   18  
...  
741      16 female <NA>   FALSE 0.550000  1.37   18  
...  
768      16 female large   FALSE 0.423360  1.82   19
```

- Okay, we have a large snail, so replace accordingly

```
> myData$Size[which(is.na(myData$Size))] = "large"
```

Replace last missing values with the average

```
> myData[which(is.na(myData$Temp)),]  
  Snail.ID    Sex  Size Feeding Distance Depth Temp  
587      13 female large   FALSE      0.60   1.41  NA  
608      13 female large   FALSE      0.20   1.05  NA  
637      14 female large   FALSE      0.38   1.72  NA  
682      15 female large    TRUE      0.58   1.21  NA  
701      15 female large    TRUE      0.98   1.43  NA  
743      16 female large   FALSE      0.45   1.78  NA  
> mean(myData$Temp) #Be careful here, you have to say that you want NAs ignored!  
[1] NA  
> mean(myData$Temp, na.rm = T)  
[1] 19.49344  
> myData$Temp[which(is.na(myData$Temp))] = mean(myData$Temp, na.rm = T)
```

Replace last missing value by correcting it

```
> myData$Depth[which(myData$Depth > 2)] = 1.62
> summary(myData)
```

	Snail.ID	Sex	Size	Feeding	Distance	Depth	Temp
1	: 48	female:384	large:384	Mode :logical	Min. :0.0000	Min. :1.000	Min. :18.00
2	: 48	male :384	small:384	FALSE:502	1st Qu.:0.2775	1st Qu.:1.260	1st Qu.:19.00
3	: 48			TRUE :266	Median :0.5100	Median :1.510	Median :19.00
4	: 48				Mean :0.5122	Mean :1.507	Mean :19.49
5	: 48				3rd Qu.:0.7500	3rd Qu.:1.760	3rd Qu.:20.00
6	: 48				Max. :1.0000	Max. :2.000	Max. :21.00
	(Other):480						

Checking for Duplicates

```
> myData[which(duplicated(myData)),]  
   Snail.ID    Sex  Size Feeding Distance Depth Temp  
733      16 female large   FALSE      0.25  1.13   21  
  
> myData<- myData[!duplicated(myData), ]
```

- Be careful when removing duplicates: Make sure they are not a legitimate entry!
- You can also use `unique()` to do the same:
`myData<- unique(myData)`

Overview of important functions

read.table() read.csv()	Import data saved as text file (.txt) or as comma separated format (.csv) into R
is.na()	indicates which elements are missing (NA)
str()	provides an overview of an object
summary()	returns basic statistical summary for variables
duplicated()	indicates which rows are duplicates
unique()	removes duplicate elements
which()	allows to select specific elements from a data frame
sort()	sorts values in a vector or a factor
order()	sorts data by a set of variables at the same time
head()	returns the first records of an object
tail()	returns the last records of an object

#sdu.dk

Data cleaning Checklist

Data Constraints Problems

Data type constraints

Ensuring that different columns have the correct data type before beginning analysis.

Example A revenue_usd column that is a string, and not a numeric data type.

revenue_usd	revenue_usd
\$1,000	1,000
\$3,210	3,210

- ✓ Convert to the correct data type

Data range constraints

Ensuring that different columns have the correct range. This is especially the case for columns that have limits.

Example A gpa column should be constrained to [0.0, 4.0]

gpa
3.1
4.0
5.1

- ✓ Check for typos, like a decimal point in the wrong place.
- ✓ Drop rows where data points break range constraints
- ✓ Set the data point that breaks range constraints to the maximum, or minimum
- ✓ Treat the data point that breaks range constraints to missing, and impute it

Uniqueness Constraints

Ensuring that there are no exact or almost exact duplicates within your rows.

Example A duplicate row where the name and phone_number columns are identical, but not the height_cm column

name	height_cm	phone_number
Carl Rosseel	177	(555) 200-5598
Carl Rosseel	178	(555) 200-5598

- ✓ Keep only one of the exact duplicate rows
- ✓ Merge rows that have non-exact duplicate rows

➤ https://images.datacamp.com/image/upload/v1654855370/Marketing/Blog/Data_Cleaning_Checklist.pdf

Data cleaning Checklist

Text and Categorical Data Problems

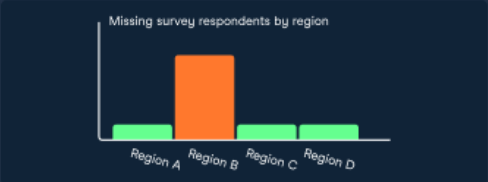
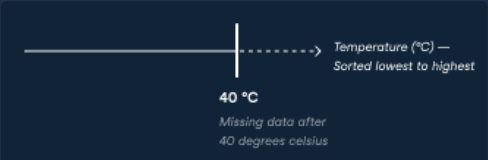
Membership constraints for categorical data Ensuring that categorical columns have correct and consistent categories	Example Two different entries for "New York" in the city column <table><thead><tr><th>name</th><th>city</th></tr></thead><tbody><tr><td>Carl Rosseel</td><td>New York</td></tr><tr><td>Sara Billen</td><td>New York City</td></tr></tbody></table>	name	city	Carl Rosseel	New York	Sara Billen	New York City	✓ Drop rows that are affected by inconsistent categories ✓ Remap inconsistent categories to the correct category name ✓ Infer categories based on other data points if it's not clear how it should be remapped			
name	city										
Carl Rosseel	New York										
Sara Billen	New York City										
Length violation for text data Ensuring that text columns that follow a specific standard have the same string length	Example A phone_number column that is 9 characters instead of 14 <table><thead><tr><th>name</th><th>height_cm</th><th>phone_number</th></tr></thead><tbody><tr><td>Carl Rosseel</td><td>177</td><td>(555) 200-5598</td></tr><tr><td>Carl Rosseel</td><td>178</td><td>(555) 200-5598</td></tr></tbody></table>	name	height_cm	phone_number	Carl Rosseel	177	(555) 200-5598	Carl Rosseel	178	(555) 200-5598	✓ Drop rows that are affected by length violation ✓ Set affected observations to missing
name	height_cm	phone_number									
Carl Rosseel	177	(555) 200-5598									
Carl Rosseel	178	(555) 200-5598									
Text data inconsistent formatting Ensuring that text columns that follow a specific standard have the same string formatting	Example A phone_number number column that contains different phone number formats <table><thead><tr><th>name</th><th>height_cm</th><th>phone_number</th></tr></thead><tbody><tr><td>Carl Rosseel</td><td>177</td><td>(555) 200-5598</td></tr><tr><td>Carl Rosseel</td><td>178</td><td>(555) 200-5598</td></tr></tbody></table>	name	height_cm	phone_number	Carl Rosseel	177	(555) 200-5598	Carl Rosseel	178	(555) 200-5598	✓ Standardize formatting for affected observations ✓ Drop rows that are affected by the inconsistency
name	height_cm	phone_number									
Carl Rosseel	177	(555) 200-5598									
Carl Rosseel	178	(555) 200-5598									

➤ https://images.datacamp.com/image/upload/v1654855370/Marketing/Blog/Data_Cleaning_Checklist.pdf

Data cleaning Checklist

➤ https://images.datacamp.com/image/upload/v1654855370/Marketing/Blog/Data_Cleaning_Checklist.pdf

Data cleaning Checklist

Missing Data Problems		
Missing Completely at Random Data When there is no systematic relationship between missing values and other values within the dataset	 There is no observed relationship between missing data and other values within the dataset	✓ Drop missing rows ✓ Impute missing rows with measures of centrality such as median or mean ✓ Impute missing rows with algorithmic, machine-learning based approaches ✓ Collect new data points and features
Missing at Random Data When there is a systematic relationship between missing data and other observed values	Missing census data from a specific region, because the postal service doesn't have full coverage in that region 	
Missing Not at Random Data When there is a systematic relationship between missing data and other unobserved values	Temperature readings from a sensor missing because temperature was too low, or high 	

➤ https://images.datacamp.com/image/upload/v1654855370/Marketing/Blog/Data_Cleaning_Checklist.pdf

Q&A

contact: roettger@imada.sdu.dk